

```

import os
import time
import requests
from datetime import datetime

# =====
# CONFIGURATION
# =====
import sys
TELEGRAM_TOKEN = sys.argv[1] if len(sys.argv) > 1 else os.environ.get(str.__new__(
CHAT_ID          = os.environ.get(str.__new__(str, chr(67)+chr(72)+chr(65)+chr(84)+
GROQ_API_KEY     = os.environ.get(str.__new__(str, chr(71)+chr(82)+chr(79)+chr(81)+

# =====
# PARAMETRES
# =====
SCAN_INTERVAL      = 120
TOP_N_COINS        = 500
COINS_PER_PAGE     = 100
BUY_THRESHOLD_24H  = 4.0
SELL_THRESHOLD_24H = -4.0
BUY_THRESHOLD_1H   = 2.5
SELL_THRESHOLD_1H  = -2.5
RSI_BUY            = 33
RSI_SELL           = 70
ALERT_COOLDOWN     = 1800
TOP_PICKS_INTERVAL = 3600

# =====
# STATE
# =====
last_alert_time = {}
last_top_picks  = 0
last_update_id  = 0
all_prices_cache = {}

# =====
# GROQ ? Appel IA gratuit
# =====
def ask_groq(prompt: str, max_tokens: int = 300) -> str:
    try:
        r = requests.post(
            "https://api.groq.com/openai/v1/chat/completions",
            headers={

```

```

        "Authorization": f"Bearer {GROQ_API_KEY}",
        "Content-Type": "application/json"
    },
    json={
        "model": "llama3-8b-8192",
        "max_tokens": max_tokens,
        "messages": [
            {
                "role": "system",
                "content": "Tu es CryptoMind, expert en cryptomonnaies. R
            },
            {
                "role": "user",
                "content": prompt
            }
        ]
    },
    timeout=20
)
data = r.json()
return data["choices"][0]["message"]["content"]
except Exception as e:
    print(f"[GROQ ERROR] {e}")
    return "Analyse IA indisponible."

# =====
# TELEGRAM ? Envoi
# =====
def send_telegram(text: str, chat_id: str = None):
    cid = chat_id or CHAT_ID
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
    chunks = [text[i:i+4000] for i in range(0, len(text), 4000)]
    for chunk in chunks:
        try:
            requests.post(url, json={
                "chat_id": cid,
                "text": chunk,
                "parse_mode": "HTML",
                "disable_web_page_preview": True
            }, timeout=10)
            time.sleep(0.3)
        except Exception as e:
            print(f"[TELEGRAM ERROR] {e}")

# =====
# TELEGRAM ? Lire les commandes
# =====

```

```

def get_updates():
    global last_update_id
    try:
        r = requests.get(
            f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/getUpdates",
            params={"offset": last_update_id + 1, "timeout": 5},
            timeout=10
        )
        updates = r.json().get("result", [])
        if updates:
            last_update_id = updates[-1]["update_id"]
        return updates
    except:
        return []

def process_commands(updates):
    for update in updates:
        msg = update.get("message", {})
        text = msg.get("text", "").strip()
        chat_id = str(msg.get("chat", {}).get("id", ""))
        if not text or not chat_id:
            continue
        print(f"[CMD] '{text}'")
        if text.startswith("/start") or text.startswith("/aide"):
            handle_start(chat_id)
        elif text.startswith("/top"):
            handle_top(chat_id, text)
        elif text.startswith("/analyse "):
            handle_analyse(chat_id, text.replace("/analyse ", "").strip().upper())
        elif text.startswith("/fraude ") or text.startswith("/check "):
            handle_fraud(chat_id, text.split(" ", 1)[1].strip())
        elif text.startswith("/meilleur"):
            handle_best_now(chat_id)
        elif text.startswith("/march"):
            handle_market(chat_id)
        elif text.startswith("/scan"):
            handle_scan_status(chat_id)
        else:
            handle_free_question(chat_id, text)

# =====
# COMMANDES
# =====

def handle_start(chat_id):
    send_telegram(""? <b>CryptoMind Bot ? Commandes</b>

? <b>Analyses</b>

```

/analyse BTC ? Analyse d'une crypto
/top 10 ? Top cryptos du moment
/meilleur ? Meilleure crypto maintenant
/march? ? ?tat du march?

?? S?curit?

/fraude 0x... ? V?rifie une adresse

? /scan ? Statut de la surveillance

? Pose n'importe quelle question !

Ex: "C'est quoi Solana ?"

?? <i>Pas un conseil financier</i>""", chat_id)

```
def handle_top(chat_id, text):
    parts = text.split()
    n = 5
    for p in parts:
        if p.isdigit():
            n = min(int(p), 20)
            break
    if not all_prices_cache:
        send_telegram("? Scan en cours, r?essaie dans 30s.", chat_id)
        return
    scored = sorted(all_prices_cache.values(), key=lambda d: score_coin(d), reverse=True)
    lines = []
    for i, d in enumerate(scored, 1):
        e = "?" if d["change_24h"] >= 0 else "-"
        lines.append(f"{i}. {e} <b>{d['symbol']}</b> ? {fmt_price(d['price'])}\n")
    msg = f"? <b>TOP {n} CRYPTOS</b>\n\n<i>{len(all_prices_cache)} cryptos analys?e"
    send_telegram(msg, chat_id)
```

```
def handle_analyse(chat_id, symbol):
    send_telegram(f"? Analyse de <b>{symbol}</b>...", chat_id)
    coin = next((d for d in all_prices_cache.values() if d["symbol"] == symbol), None)
    if not coin:
        send_telegram(f"? <b>{symbol}</b> non trouv?. V?rifie le symbole (ex: BTC)", chat_id)
        return
    send_telegram(deep_analyze_coin(coin), chat_id)
```

```
def handle_fraud(chat_id, address):
    if not address or len(address) < 10:
        send_telegram("? Fournis une adresse valide.\nEx: /fraude 0x742d35...", chat_id)
        return
    send_telegram(f"? Analyse de l'adresse...\n<code>{address}</code>", chat_id)
    send_telegram(check_fraud(address), chat_id)
```

```

def handle_best_now(chat_id):
    send_telegram("? Recherche de la meilleure opportunit?...", chat_id)
    if not all_prices_cache:
        send_telegram("? Scan en cours, r?essaie dans 30s.", chat_id)
        return
    send_telegram(find_best_opportunity(), chat_id)

def handle_market(chat_id):
    if not all_prices_cache:
        send_telegram("? Donn?es non disponibles.", chat_id)
        return
    coins = list(all_prices_cache.values())
    top5_1h = sorted(coins, key=lambda d: d["change_1h"], reverse=True)[:5]
    top5_24h = sorted(coins, key=lambda d: d["change_24h"], reverse=True)[:5]
    bullish = sum(1 for c in coins if c["change_24h"] > 0)
    sent = "? Haussier" if bullish > len(coins)/2 else "? Baissier"
    def line(d, k): return f" {'?' if d[k]>=0 else '?'} <b>{d['symbol']}

```

```

        r.raise_for_status()
    for c in r.json():
        result[c["id"]] = {
            "id": c["id"],
            "name": c["name"],
            "symbol": c["symbol"].upper(),
            "price": c["current_price"] or 0,
            "change_1h": c.get("price_change_percentage_1h_in_currency")
            "change_24h": c.get("price_change_percentage_24h") or 0,
            "change_7d": c.get("price_change_percentage_7d_in_currency")
            "volume": c.get("total_volume") or 0,
            "market_cap": c.get("market_cap") or 0,
            "high_24h": c.get("high_24h") or 0,
            "low_24h": c.get("low_24h") or 0,
            "rank": c.get("market_cap_rank") or 999,
        }
    print(f"[?] Page {page}/5 ? {len(result)} coins")
    time.sleep(1.5)
except Exception as e:
    print(f"[COINGECKO PAGE {page}] {e}")
    time.sleep(5)

return result

# =====
# SCORE
# =====

def score_coin(d):
    s = 50
    s += min(d["change_1h"] * 3, 15) if d["change_1h"] > 0 else max(d["change_1h"]
    s += min(d["change_24h"] * 2, 12) if d["change_24h"] > 0 else max(d["change_2
    s += min(d["change_7d"], 8) if d["change_7d"] > 0 else 0
    if d["volume"] > 1_000_000_000: s += 8
    elif d["volume"] > 100_000_000: s += 4
    if d["market_cap"] > 10_000_000_000: s += 5
    elif d["market_cap"] < 10_000_000: s -= 10
    if d["rank"] <= 10: s += 5
    elif d["rank"] > 200: s -= 5
    return max(0, min(100, int(s)))

def estimate_rsi(c24, c1):
    return max(10, min(90, int(50 + c24 * 2.5 + c1 * 5)))

def fmt_price(p):
    if p >= 1000: return f"${p:,.2f}"
    elif p >= 1: return f"${p:.4f}"
    elif p >= 0.01: return f"${p:.5f}"
    else: return f"${p:.8f}"

```

```

# =====
# ANALYSE PROFONDE
# =====
def deep_analyze_coin(d):
    rsi = estimate_rsi(d["change_24h"], d["change_1h"])
    score = score_coin(d)
    signal = "ACHAT" if score >= 65 else "VENTE" if score <= 35 else "NEUTRE"
    prompt = (f"Analyse {d['name']} ({d['symbol']}) :\n"
              f"Prix: {fmt_price(d['price'])} | Rang: #{d['rank']}\n"
              f"1h: {d['change_1h']:+.2f}% | 24h: {d['change_24h']:+.2f}% | 7j: {d['change_7j']:+.2f}%\n"
              f"Volume: ${d['volume']:, .0f} | RSI: {rsi} | Score: {score}/100\n"
              f"Plus haut 24h: {fmt_price(d['high_24h'])} | Plus bas: {fmt_price(d['low_24h'])}\n"
              f"Donne : signal ACHETER/VENDRE/ATTENDRE, prix d'entr e, objectif,
    ai = ask_groq(prompt, 400)
    e = "?" if signal == "ACHAT" else "?" if signal == "VENTE" else "?"
    return (f"? <b>{d['name']} ({d['symbol']})</b> ? Rang #{d['rank']}\n\n"
           f"? Prix: <b>{fmt_price(d['price'])}</b>\n"
           f"? 1h: {d['change_1h']:+.2f}% 24h: {d['change_24h']:+.2f}% 7j: {d['change_7j']:+.2f}%\n"
           f"? RSI: {rsi} Score: {score}/100\n\n"
           f"{e} <b>Signal: {signal}</b>\n\n"
           f"? <b>Analyse IA:</b>\n{ai}\n\n"
           f"<i>?? Pas un conseil financier</i>")

# =====
# MEILLEURE OPPORTUNIT 
# =====
def find_best_opportunity():
    coins = list(all_prices_cache.values())
    best_1h = sorted(coins, key=lambda d: d["change_1h"], reverse=True)[0]
    best_24h = sorted(coins, key=lambda d: d["change_24h"], reverse=True)[0]
    best_sc = sorted(coins, key=lambda d: score_coin(d), reverse=True)[0]
    prompt = (f"3 meilleures cryptos d'tect es parmi 500 :\n"
              f"Top 1h: {best_1h['name']} ({best_1h['symbol']}) {best_1h['change_1h']:+.1f}%\n"
              f"Top 24h: {best_24h['name']} ({best_24h['symbol']}) {best_24h['change_24h']:+.1f}%\n"
              f"Top score: {best_sc['name']} ({best_sc['symbol']}) score {score_coin(best_sc)}/100\n"
              f"Laquelle choisir pour investir dans l'heure ? Prix d'entr e et
    ai = ask_groq(prompt, 300)
    return (f"? <b>MEILLEURE OPPORTUNIT  MAINTENANT</b>\n"
           f"<i>{len(all_prices_cache)} cryptos analys es</i>\n\n"
           f"? Top 1h: <b>{best_1h['symbol']}</b> {best_1h['change_1h']:+.1f}%\n"
           f"? Top 24h: <b>{best_24h['symbol']}</b> {best_24h['change_24h']:+.1f}%\n"
           f"? Top score: <b>{best_sc['symbol']}</b> {score_coin(best_sc)}/100\n"
           f"? <b>Recommandation IA:</b>\n{ai}\n\n"
           f"<i>?? Court terme = risque ?lev </i>")

# =====

```

```

# DETECTION FRAUDE
# =====
def check_fraud(address):
    is_eth = address.startswith("0x") and len(address) == 42
    is_sol = len(address) in [43, 44] and not address.startswith("0x")
    is_btc = address.startswith(("1", "3", "bc1")) and 25 <= len(address) <= 62
    chain = "Ethereum/BSC" if is_eth else "Solana" if is_sol else "Bitcoin" if i

    flags = []
    goplus_result = "Non vérifié?"

    if is_eth:
        try:
            r = requests.get(
                f"https://api.gopluslabs.io/api/v1/address_security/{address}",
                timeout=10
            )
            gp = r.json()
            if gp.get("code") == 1:
                res = gp.get("result", {})
                checks = {
                    "cybercrime": "? Cybercriminalité?",
                    "money_laundering": "? Blanchiment",
                    "phishing_activities": "? Phishing",
                    "stealing_attack": "? Vol détecté?",
                    "blacklist_doubt": "? Liste noire",
                    "darkweb_transactions": "? Darkweb",
                    "sanctioned": "? SANCTIONNÉ?",
                }
                for key, label in checks.items():
                    if res.get(key) == "1":
                        flags.append(label)
                goplus_result = "DANGEREUX" if flags else "Aucun signal connu"
            except:
                goplus_result = "API indisponible"

    prompt = (f"Analyse cette adresse crypto pour détecter une arnaque.\n"
              f"Adresse: {address}\nRéseau: {chain}\n"
              f"Signaux GoPlus: {goplus_result}\nFlags détectés: {flags if flags else ''}\n"
              f"Verdict: ARNAQUE / SUSPECT / LÉGITIME. Niveau risque. Que faire.")
    ai = ask_groq(prompt, 300)
    flags_text = "\n".join(flags) if flags else "? Aucun signal d'arnaque"

    return (f"? <b>ANALYSE FRAUDE</b>\n"
            f"<code>{address[:20]}...{address[-6:]}</code>\n\n"
            f"? Réseau: <b>{chain}</b>\n"
            f"? GoPlus: <b>{goplus_result}</b>\n")

```



```

        f"{flags_text}\n\n"
        f"? <b>Verdict IA:</b>\n{ai}\n\n"
        f"? V?rifie aussi: etherscan.io")

# =====
# ALERTES AUTOMATIQUES
# =====
def check_alerts(prices):
    now = time.time()
    for coin_id, d in prices.items():
        c1h = d["change_1h"]
        c24h = d["change_24h"]
        rsi = estimate_rsi(c24h, c1h)
        if now - last_alert_time.get(coin_id, 0) < ALERT_COOLDOWN:
            continue
        signal = None
        reasons = []
        if c1h >= BUY_THRESHOLD_1H and c24h > 0:
            signal = "buy"; reasons.append(f"? +{c1h:.1f}% en 1h")
        if c24h >= BUY_THRESHOLD_24H:
            signal = "buy"; reasons.append(f"? +{c24h:.1f}% en 24h")
        if rsi <= RSI_BUY:
            signal = "buy"; reasons.append(f"? RSI bas ({rsi})")
        if c1h <= SELL_THRESHOLD_1H and c24h < 0:
            signal = "sell"; reasons = [f"? {c1h:.1f}% en 1h"]
        if c24h <= SELL_THRESHOLD_24H:
            signal = "sell"; reasons = [f"? {c24h:.1f}% en 24h"]
        if rsi >= RSI_SELL:
            signal = "sell"; reasons = [f"? RSI ?lev? ({rsi})"]
        if not signal:
            continue

    prompt = (f"{d['name']} ({d['symbol']}) signal {signal.upper()}. "
              f"Prix: {fmt_price(d['price'])} | 1h: {c1h:+.1f}% | 24h: {c24h:}
              f"En 2 phrases: pourquoi et que faire maintenant ?")
    ai = ask_groq(prompt, 150)
    icon = "?" if signal == "buy" else "?"
    lbl = "ACHAT" if signal == "buy" else "VENTE"
    e = "?" if signal == "buy" else "?"
    msg = (f"{icon} <b>{lbl} ? {d['name']} ({d['symbol']})</b> {e}\n\n"
          f"? Prix: <b>{fmt_price(d['price'])}</b>\n"
          f"1h: {c1h:+.1f}% 24h: {c24h:+.1f}% 7j: {d['change_7d']:+.1f}%\n"
          f"<b>Raisons:</b>\n" + "\n".join(f" {r}" for r in reasons) +
          f"\n\n? {ai}\n\n"
          f"<i>? /analyse {d['symbol']} pour plus de d?tails\n"
          f"?? Pas un conseil financier</i>")
    send_telegram(msg)

```

```

        last_alert_time[coin_id] = now
        print(f"[?] {lbl}: {d['name']}")

# =====
# RAPPORT HORAIRE
# =====

def send_hourly_picks():
    global last_top_picks
    if time.time() - last_top_picks < TOP_PICKS_INTERVAL or not all_prices_cache:
        return
    last_top_picks = time.time()
    top3 = sorted(all_prices_cache.values(), key=lambda d: score_coin(d), reverse=True)
    lines = [f"{i}. ? <b>{d['symbol']}</b> ? {fmt_price(d['price'])}\n    1h: {d['1h']}"
              for i, d in enumerate(top3, 1)]
    send_telegram(f"? <b>RAPPORT HORAIRE ? {datetime.now().strftime('%H:%M')}</b>"
                  "\n\n".join(lines) +
                  "\n? /meilleur pour la recommandation IA")
    print(f"[?] Rapport horaire envoy?")

# =====
# D?MARRAGE
# =====

def main():
    print("=" * 50)
    print("  CryptoMind Bot (Groq) ? D?marrage")
    print("=" * 50)
    send_telegram(f"? <b>CryptoMind Bot ? D?MARR? ?</b>\n\n"
                  "? Scan Top 500 cryptos en cours...\n"
                  "? Propuls? par Groq AI (gratuit)\n\n"
                  "? Tape /aide pour les commandes\n"
                  "? Tape /meilleur pour la meilleure crypto")
    scan = 0
    while True:
        scan += 1
        print(f"\n[SCAN #{scan}] {datetime.now().strftime('%H:%M:%S')}")
        updates = get_updates()
        if updates:
            process_commands(updates)
        prices = fetch_top_500()
        if prices:
            all_prices_cache.update(prices)
            check_alerts(prices)
            send_hourly_picks()
        else:
            time.sleep(30)
            continue
    print(f"[?] Prochain scan dans {SCAN_INTERVAL}s")

```

```
time.sleep(SCAN_INTERVAL)
```

```
if __name__ == "__main__":  
    main()
```